## 1. Load and Explore the Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# Load the dataset
df = pd.read_csv('/content/dbscan_dataset.csv')
display(df.head())
print("Dataset Shape:", df.shape)
```

| | Feature1 | Feature2 |
|---|---|---|
| **0** | -7.155244 | -7.390016 |
| **1** | -7.395875 | -7.110843 |
| **2** | -2.015671 | 8.281780 |
| **3** | 4.509270 | 2.632436 |
| **4** | -8.102502 | -7.484961 |

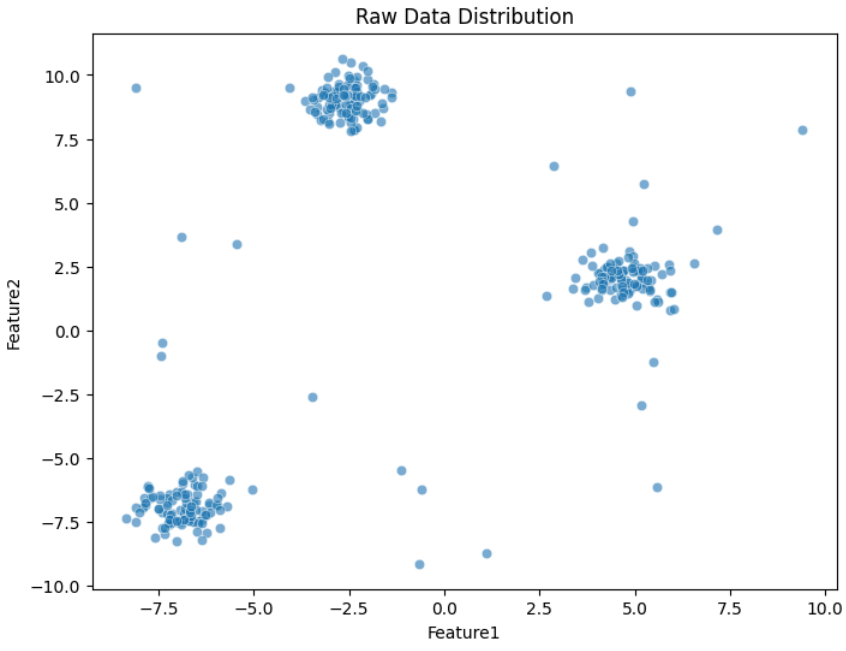
```
Dataset Shape: (320, 2)
```

## 2. Visualize the Unclustered Data

Let's plot the raw data points to understand their spatial distribution before clustering.

```
# Assuming the dataset has at least two numerical columns for plotting
x_col = df.columns[0]
y_col = df.columns[1]

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x=x_col, y=y_col, alpha=0.6)
plt.title('Raw Data Distribution')
plt.xlabel(x_col)
plt.ylabel(y_col)
plt.show()
```




## 3. Preprocess and Apply DBSCAN

We standard scale the features first, as DBSCAN is sensitive to feature scales (Euclidean distance metric).

```
# Scale the features
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df[[x_col, y_col]])

# Apply DBSCAN
# We start with some reasonable default parameters (eps=0.3, min_samples=5)
dbscan = DBSCAN(eps=0.3, min_samples=5)
dbscan_labels = dbscan.fit_predict(scaled_features)

# Add the cluster labels to our DataFrame
df['Cluster'] = dbscan_labels
```

```
# Calculate number of clusters and noise points
n_clusters_ = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
n_noise_ = list(dbscan_labels).count(-1)

print(f'Estimated number of clusters: {n_clusters_}')
print(f'Estimated number of noise points: {n_noise_}')
```

```
Estimated number of clusters: 3
Estimated number of noise points: 16
```

## 4. Visualize the Clustering Results

We color-code the clusters and plot noise points explicitly (labeled as `-1`).

```
plt.figure(figsize=(10, 8))
# Use a distinct palette where noise (-1) is easily visible
sns.scatterplot(data=df, x=x_col, y=y_col, hue='Cluster', palette='viridis', legend='full', alpha=0.8)
plt.title(f'DBSCAN Clustering (Clusters: {n_clusters_}, Noise Points: {n_noise_})')
plt.xlabel(x_col)
plt.ylabel(y_col)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

